ASSIGNMENT 17-4

Task 1 – Smart City Traffic Sensor Data Cleaning

Scenario

A Smart City project collects real-time traffic sensor data from multiple intersections. However, the dataset contains inconsistent formats, missing values, and corrupted readings.

```python
import pandas as pd
import os
if not os.path.exists("traffic_sensor_data.csv"):
    print("traffic_sensor_data.csv not found. Creating a dummy dataset for demonstration.")
    dummy_data = {
        'timestamp': pd.to_datetime(['2023-01-01 08:00:00', '2023-01-01 08:05:00']),
        'intersection': ['Main St', 'Oak Ave', 'Main St ', ' Elm  St', 'Oak Ave'],
        'vehicle_count': [10, 15, 8, None, 20],
        'avg_speed': [45.5, 30.2, 50.0, 35.8, None]
    }
    dummy_df = pd.DataFrame(dummy_data)
    dummy_df.to_csv("traffic_sensor_data.csv", index=False)
    print("Dummy traffic_sensor_data.csv created.")
df = pd.read_csv("traffic_sensor_data.csv")
original_df = df.copy()
df['vehicle_count'].fillna(df['vehicle_count'].median(), inplace=True)
df['avg_speed'].fillna(df['avg_speed'].median(), inplace=True)
df.drop_duplicates(inplace=True)
df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')
df = df.dropna(subset=['timestamp'])
df = df[(df['vehicle_count'] >= 0) & (df['avg_speed'] >= 0)]
df['intersection'] = df['intersection'].str.upper().str.strip()
df.reset_index(drop=True, inplace=True)
df.to_csv("cleaned_traffic_sensor_data.csv", index=False)
```

```python
df.reset_index(drop=True, inplace=True)
df.to_csv("cleaned_traffic_sensor_data.csv", index=False)

print(df.head())
print("\nCleaning Summary:")
print("Original rows:", len(original_df))
print("Cleaned rows:", len(df))
print("Duplicates removed:", len(original_df) - len(original_df.drop_duplicates()))
print("Invalid rows removed:", len(original_df) - len(df))
```

```
traffic_sensor_data.csv not found. Creating a dummy dataset for demonstration.
Dummy traffic_sensor_data.csv created.
            timestamp intersection  vehicle_count  avg_speed
0 2023-01-01 08:00:00      MAIN ST           10.0      45.50
1 2023-01-01 08:05:00      OAK AVE           15.0      30.20
2 2023-01-01 08:10:00      MAIN ST            8.0      50.00
3 2023-01-01 08:15:00      ELM  ST           12.5      35.80
4 2023-01-01 08:20:00      OAK AVE           20.0      40.65

Cleaning Summary:
Original rows: 5
Cleaned rows: 5
Duplicates removed: 0
Invalid rows removed: 0
```

EXPLANATION:
Missing values in traffic metrics are filled using median to maintain realistic patterns.
Duplicates are removed to avoid repeated sensor readings.
Timestamp conversion ensures proper time-based analysis.
Invalid readings (negative speed or vehicle count) are filtered out.
Intersection names are standardized to uppercase for consistency.

Task 2 – Movie Streaming Platform User
Behavior Data
Scenario
A streaming platform wants to analyze user watch patterns, but the raw
data is messy.
Dataset contains:
• watch_time (in minutes)
• subscription_type
• login_date
• device_type

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import os
if not os.path.exists("streaming_user_data.csv"):
    print("streaming_user_data.csv not found. Creating a dummy dataset for demonstration.")
    dummy_data = {
        'user_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        'login_date': pd.to_datetime(['2023-01-01', '2023-01-02', '2023-01-03']),
        'watch_time': [120, 90, 150, 60, 180, 200, 75, 110, 130, 95],
        'subscription_type': ['Premium', 'Free', 'Premium', 'Free', 'Premium', 'Premium', 'Free', 'Premium', 'Free', 'Premium'],
        'device_type': ['Mobile', 'Desktop', 'Tablet', 'Mobile', 'Desktop', 'Mobile', 'Tablet', 'Desktop', 'Mobile', 'Desktop']
    }
    dummy_df = pd.DataFrame(dummy_data)
    dummy_df.to_csv("streaming_user_data.csv", index=False)
    print("Dummy streaming_user_data.csv created.")

df = pd.read_csv("streaming_user_data.csv")
df['login_date'] = pd.to_datetime(df['login_date'], errors='coerce')
df = df.dropna(subset=['login_date'])
df['day_of_week'] = df['login_date'].dt.dayofweek
df['is_weekend'] = df['day_of_week'].apply(lambda x: 1 if x >= 5 else 0)
df = pd.get_dummies(df, columns=['subscription_type', 'device_type'], drop_first=True)
scaler = MinMaxScaler()
df['watch_time_normalized'] = scaler.fit_transform(df[['watch_time']])
Q1 = df['watch_time'].quantile(0.25)
Q3 = df['watch_time'].quantile(0.75)
IQR = Q3 - Q1
```

```python
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df = df[(df['watch_time'] >= lower_bound) & (df['watch_time'] <= upper_bound)]

df.reset_index(drop=True, inplace=True)
df.to_csv("processed_streaming_data.csv", index=False)
print(df.head())
```

```
streaming_user_data.csv not found. Creating a dummy dataset for demonstration.
Dummy streaming_user_data.csv created.
   user_id login_date  watch_time  day_of_week  is_weekend  \
0        1 2023-01-01         120            6           1
1        2 2023-01-02          90            0           0
2        3 2023-01-03         150            1           0
3        4 2023-01-04          60            2           0
4        5 2023-01-05         180            3           0

   subscription_type_Premium  device_type_Mobile  device_type_Tablet  \
0                       True                True               False
1                      False               False               False
2                       True               False                True
3                      False                True               False
4                       True               False               False

   watch_time_normalized
0               0.428571
1               0.214286
2               0.642857
3               0.000000
```

EXPLANATION:

- **Datetime conversion** ensures proper handling of user activity timelines.
- **Feature engineering** (day_of_week, is_weekend) helps analyze viewing behavior patterns.
- **Encoding** transforms categorical variables into numeric format for ML models.
- **Outlier removal (IQR)** eliminates unrealistic watch times.
- **Normalization** scales watch_time to a 0–1 range for consistent analysis.

Task 5 – Climate Change Research Dataset
Preparation
Scenario
Researchers are analyzing global climate data containing:
• CO2_emissions
• avg_temperature
• country
• year
• extreme_event (Yes/No)

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler
import os
if not os.path.exists("climate_data.csv"):
    print("climate_data.csv not found. Creating a dummy dataset for demonstration.")
    dummy_data = {
        'country': ['USA', 'China', 'India', 'USA', 'China', 'Germany'],
        'year': [2010, 2010, 2010, 2011, 2011, 2011],
        'CO2_emissions': [15.0, 20.0, 5.0, 15.5, None, 7.0],
        'avg_temperature': [10.2, 12.5, 25.1, 10.5, 13.0, None],
        'extreme_event': ['No', 'Yes', 'No', 'No', 'Yes', 'No']
    }
    dummy_df = pd.DataFrame(dummy_data)
    dummy_df.to_csv("climate_data.csv", index=False)
    print("Dummy climate_data.csv created.")
df = pd.read_csv("climate_data.csv")
df['CO2_emissions'].fillna(df['CO2_emissions'].median(), inplace=True)
df['avg_temperature'].fillna(df['avg_temperature'].median(), inplace=True)
df['country'].fillna(df['country'].mode()[0], inplace=True)
df.drop_duplicates(inplace=True)
df['extreme_event'] = df['extreme_event'].map({'Yes': 1, 'No': 0})
df = pd.get_dummies(df, columns=['country'], drop_first=True)
scaler = StandardScaler()
num_cols = ['CO2_emissions', 'avg_temperature']

df[num_cols] = scaler.fit_transform(df[num_cols])

Q1 = df[num_cols].quantile(0.25)
```

```python
Q3 = df[num_cols].quantile(0.75)
IQR = Q3 - Q1
df = df[~((df[num_cols] < (Q1 - 1.5 * IQR)) |
          (df[num_cols] > (Q3 + 1.5 * IQR))).any(axis=1)]
df.reset_index(drop=True, inplace=True)
df.to_csv("processed_climate_data.csv", index=False)
print(df.head())
```

```
   year  CO2_emissions  avg_temperature  extreme_event  country_Germany  \
0  2010       0.399451        -0.740145              0            False
1  2010       1.358132        -0.288198              1            False
2  2011       0.495319        -0.681195              0            False
3  2011       0.399451        -0.189949              1            False
4  2011      -1.134440        -0.288198              0             True

   country_India  country_USA
0          False         True
1          False        False
2          False         True
3          False        False
4          False        False
```

EXPLANATION:

Missing values handled using median (numerical) and mode (categorical).
Duplicates removed to ensure data integrity.
Encoding converts:

- `extreme_event` → binary (0/1)
- `country` → one-hot encoded

Standard scaling normalizes CO_2 emissions and temperature.
Outlier removal (IQR) filters extreme climate readings.